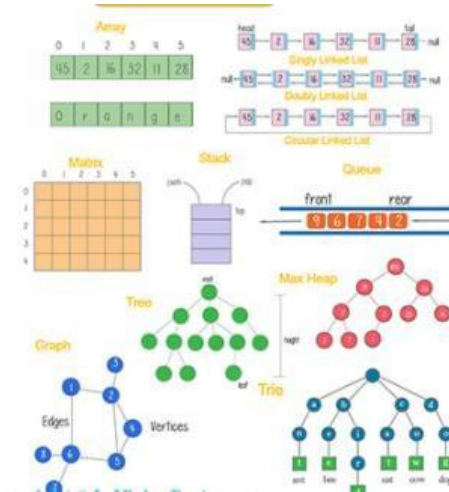
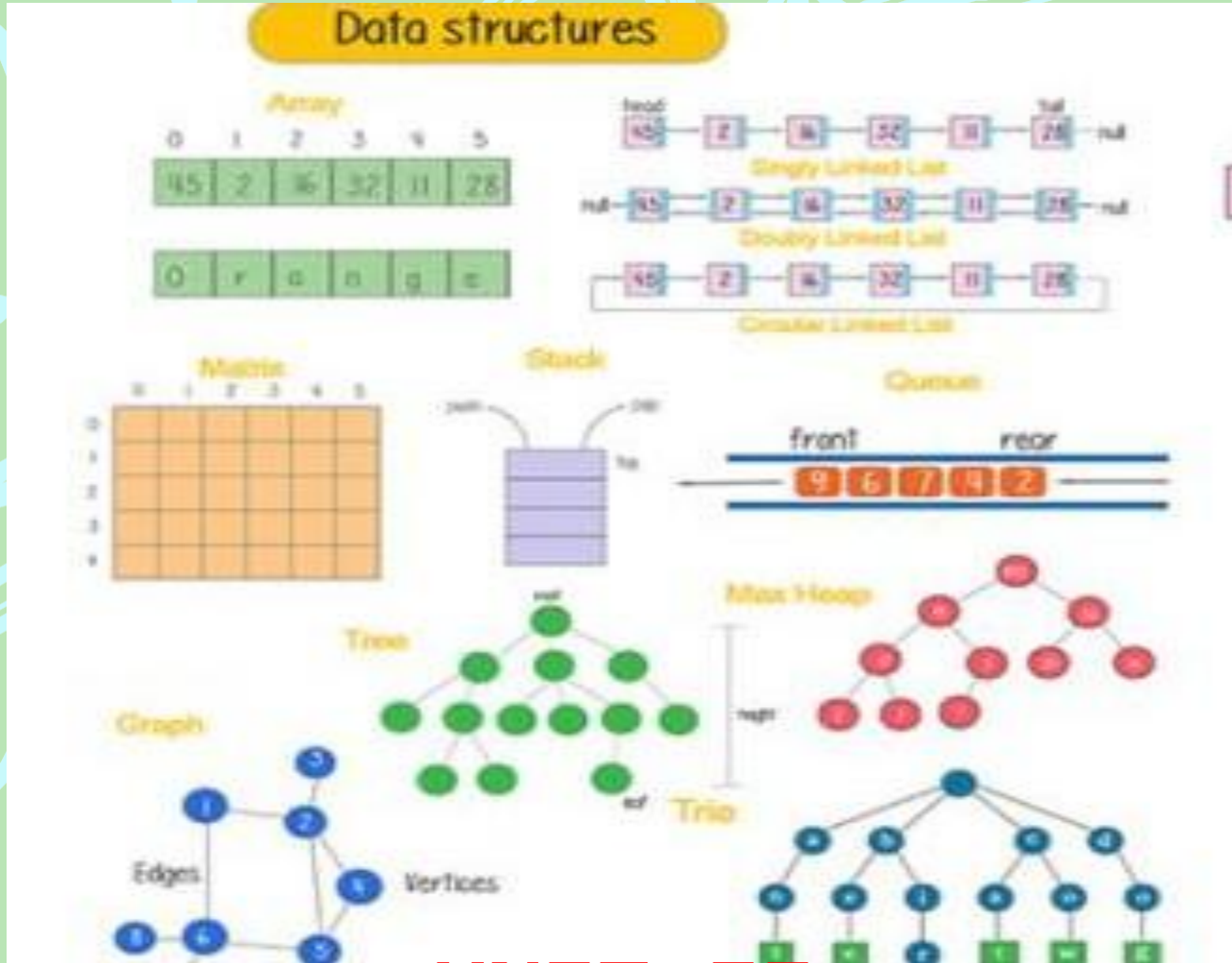


Data Structures



Dr. Elakkiya E,
Asst. Professor,
Dept. of CSE, SRM University AP





UNIT-II



LINKED LISTS

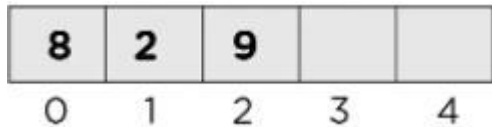
Why Linked Lists?

➤ Operations performed on Arrays

1. Create
2. Insert
3. Delete
4. Traverse
5. Search
6. Display

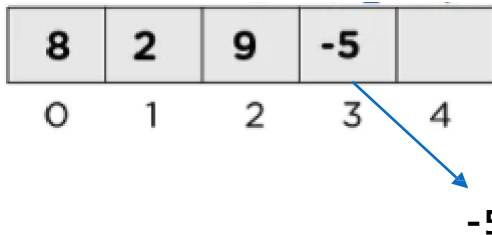
➤ Problems with Arrays

1. Memory requirements



If I store only 3 elements,
2 memory locations will be wasted

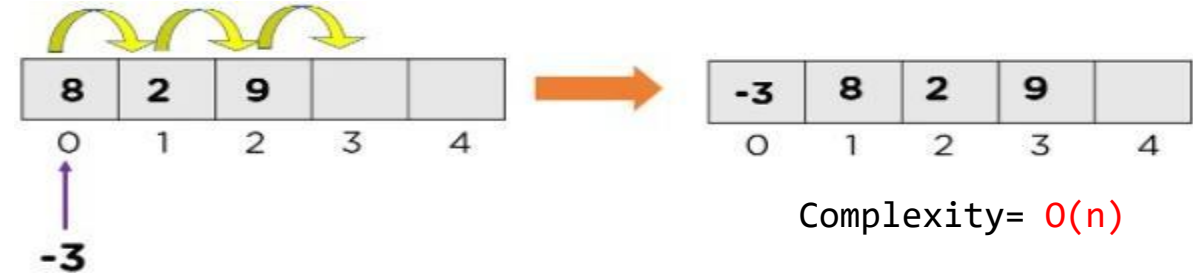
2. Cost of accessing an element



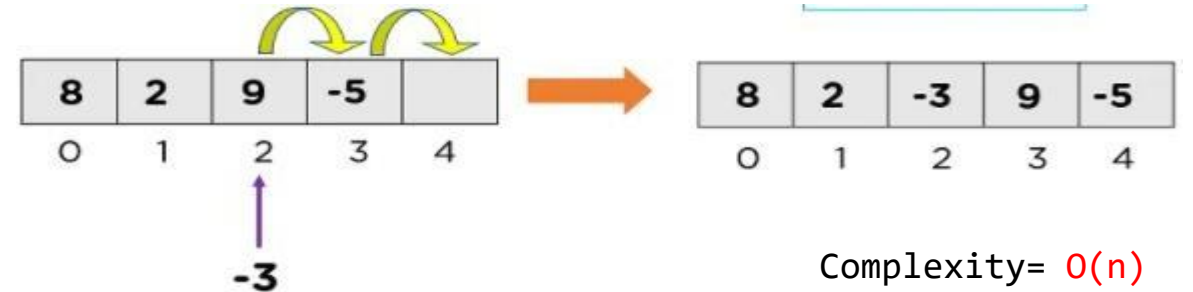
Time complexity= $O(1)$

3. Cost of Inserting an Element

Inserting at the beginning



Inserting at the middle



Inserting at the end



Why Linked Lists?

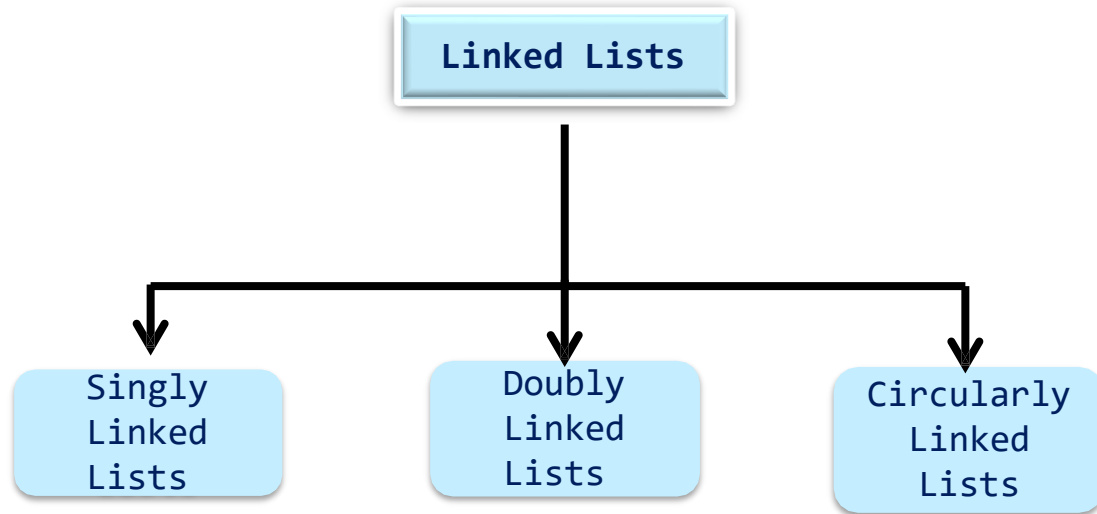
1. An array is a collection of elements of a similar data type.
2. Array elements store in a contiguous memory location.
3. Array elements are independent of each other.
4. Array works with a static memory.
5. Memory is allocated at compile-time.
6. Accessing any element in an array is faster ($O(1)$)
7. Cost is more for performing insertion, deletion, operations ($O(n)$).
8. Memory utilization is inefficient. (For example, if the size of the array is 6, and array consists of 3 elements only then the rest of the space will be unused).

1. A linked list is a collection nodes where a node consists of two parts, i.e., data and address.
2. Linked list elements can be stored anywhere in the memory or randomly stored.
3. Linked list elements are dependent on each other.
4. The Linked list works with dynamic memory.
5. Memory is allocated at run time.
6. Accessing an element in a linked list is slower ().
7. Cost is less for performing insertion, deletion operations ()
8. Memory utilization is efficient (memory can be allocated or deallocated at the run time according to our requirement).

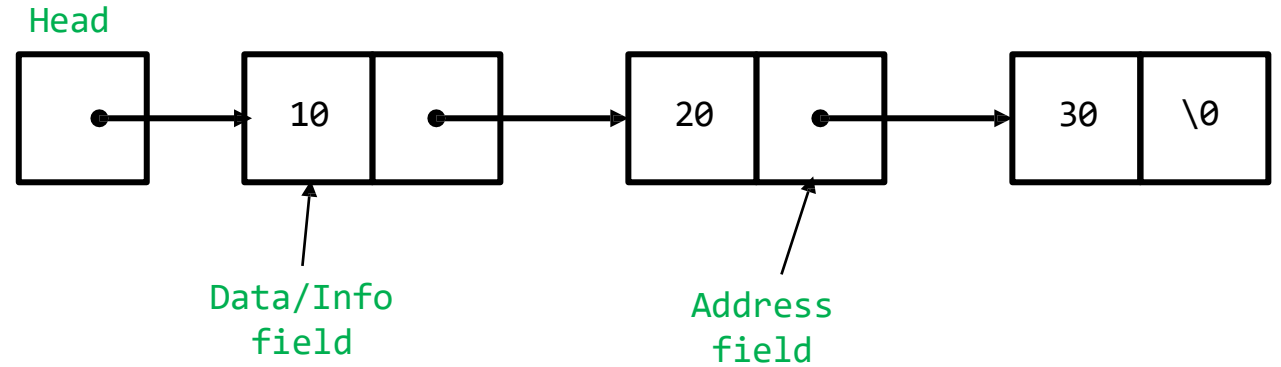
What is Linked List?

In linked list, each node consists of its own data and the address of the next node and forms a chain.

- A **linked list** is a series of connected **nodes**
- Each node contains at least
 - A piece of **data** (any type)
 - **Pointer** to the next node in the list
- **Head**: pointer to the first node
- **NULL**: The last node points to NULL



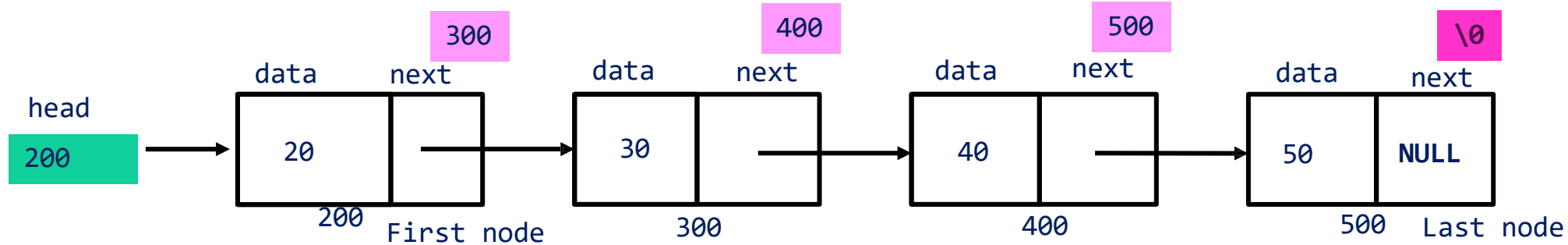
Singly Linked Lists



Operations performed on Singly Linked Lists

- Create
- Traversing
- Search
- Insertion
 - At the begin
 - At the end
 - At the middle
- Deletion
 - At the begin
 - At the end
 - At the middle
- Display

Operations on Singly Linked List



1. Creation of Linked List

- Empty Linked list is a single pointer with the value of **NULL**.

```
head = NULL;
```

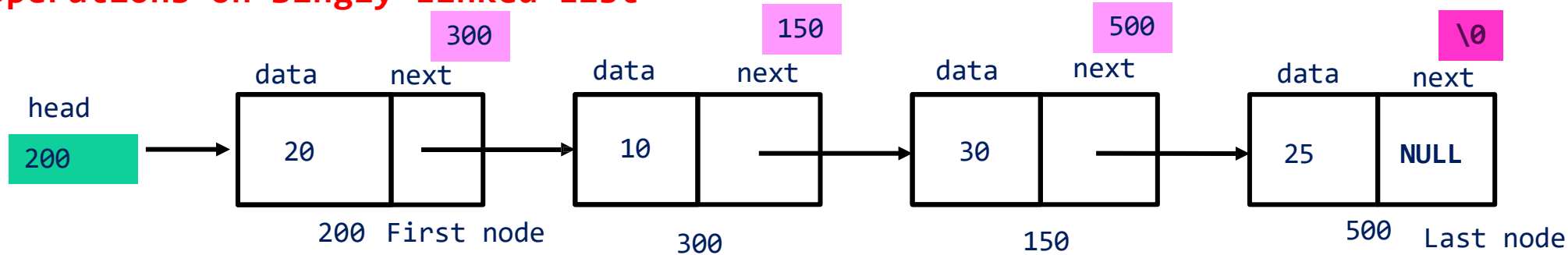
- A node can be declared as::

```
struct Node
{
    int data;
    struct Node *next;
};
struct Node *head;
```

- Create the first node and make **head** point to it.

```
head = (struct Node*)malloc(sizeof(struct Node));
```

Operations on Singly Linked List



1. Creation of Linked List

- Empty Linked list is a single pointer with the value of **NULL**.

```
head = NULL;
```

- A node can be declared as::

```
struct Node
{
    int data;
    struct Node *next;
};
```

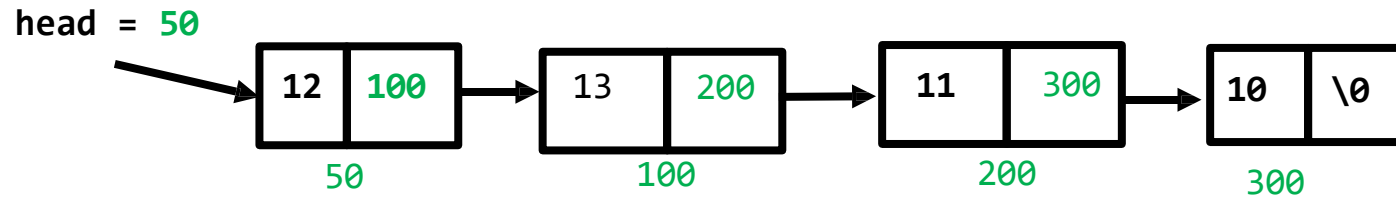
- Create the first node and make **head** point to it.

```
head = (struct Node*)malloc(sizeof(struct Node));
```

```
struct Node *create_SLL()
{
    for (i=1; i<=n; i++)
    {
        if (i == 1)
        {
            //Allocate the memory space at runtime for first node
            head = (struct Node*)malloc(sizeof(struct Node));
            ptr = head;
        }
        else
        {
            //Allocate the memory space at runtime for remaining nodes
            ptr->next = (struct Node*)malloc(sizeof(struct Node));
            ptr = ptr->next;
        }
        //Read the data into each node of SLL
        printf("\n Enter an %dth element",i);
        scanf("%d",&ptr->data);
    }

    //Assign null pointer to last node of SLL
    ptr->next = NULL;
    return(head);
}
```

Operations on Singly Linked List



2. Traversing a linked list

```
void display_SLL(struct Node *head)
{
    struct Node *ptr;
    int count=0;
    for(ptr=head; ptr!= NULL; ptr = ptr->next)
    {
        printf ("\n Node data= %d", ptr->data);
        count++;
    }
    printf ("\n");
}
```

```
void main()
{
    struct Node *head;
    head=create_SLL();
    display(head);
    search(head);
}
```

3. Searching an element in linked list

```
void search_SLL(struct Node *head)
{
    struct Node *ptr;
    int key, flag=0;

    printf("Enter an element to search:");
    scanf("%d",&key);

    for(ptr = head; ptr!= NULL; ptr = ptr->next)
    {
        if ((ptr->data)==key)
        {
            flag=1;
            break;
        }
    }

    if(flag==1)
        printf("found");
    else
        printf("not found");
}
```

Inserting node at the beginning of the List

```
struct Node *insertBeg_SLL(struct Node *head)
{
    //Step-1. create a new node

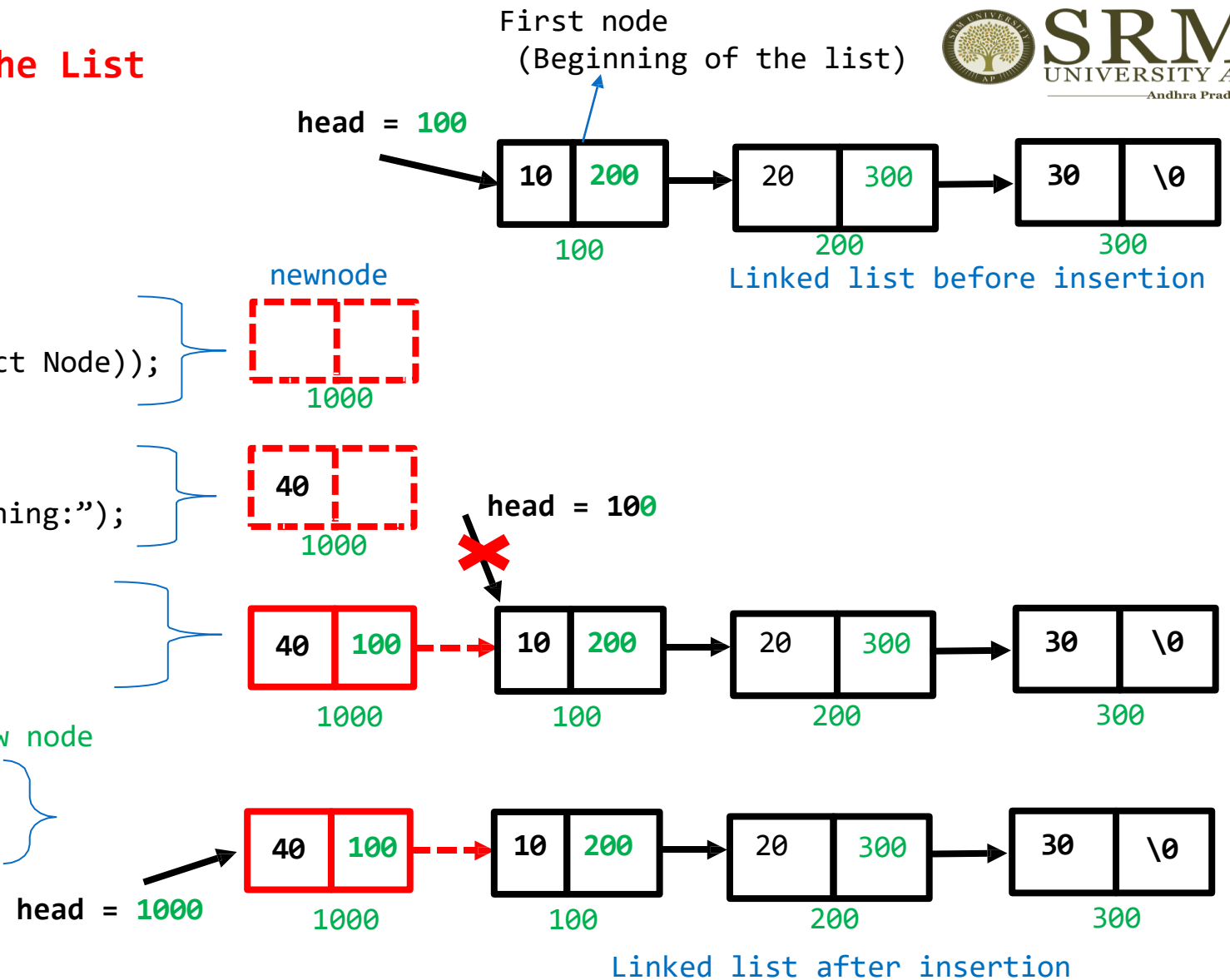
    struct Node* newNode;
    newNode = (struct Node*) malloc(sizeof(struct Node));

    //Step-2. Store the data

    printf("Enter an element to insert at beginning:");
    scanf("%d",&newNode->data);

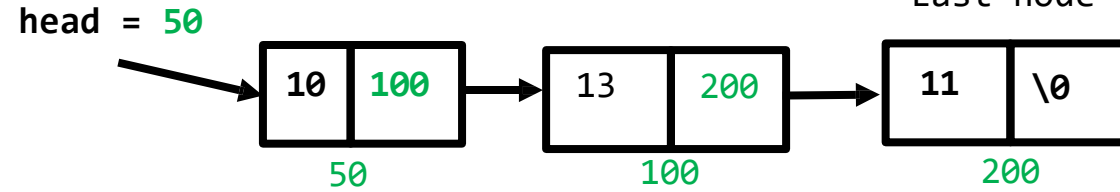
    //Step-3. Make next of new node as head
    newNode->next = head;

    //Step-4. Move the head to point to the new node
    head=newNode;
    return(head);
}
```



Inserting node at the end of the List

Last node (end of list)



```
struct Node *insertEnd_SLL(struct Node *head)
```

```
{  
    //Step-1. Create new node  
    struct Node* newnode;  
    newnode = (struct Node*) malloc(sizeof(struct Node));
```

```
    //Step-2. Store the data  
    printf("Enter an element to insert:");  
    scanf("%d",&newnode->data);
```

```
    //Step-3. New node Should be inserted last  
    // so make next of it as NULL  
    newnode->next = NULL;
```

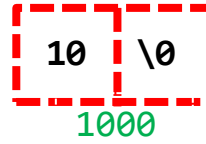
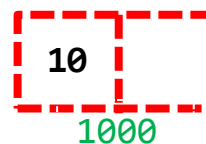
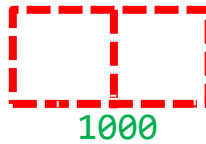
```
    //Step-4. If list is empty, then make newnode as head  
    if (head == NULL){  
        head = newnode;  
        return(head);
```

```
    }  
    //Step-5. Else traverse till the last node  
    struct Node *ptr=head;  
    while (ptr->next != NULL){  
        ptr = ptr->next;
```

```
    }  
    //Step-6. Change the next of last node  
    ptr->next = newnode;  
    return(head);
```

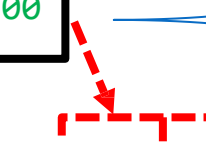
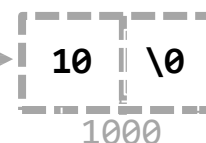
```
}
```

newnode

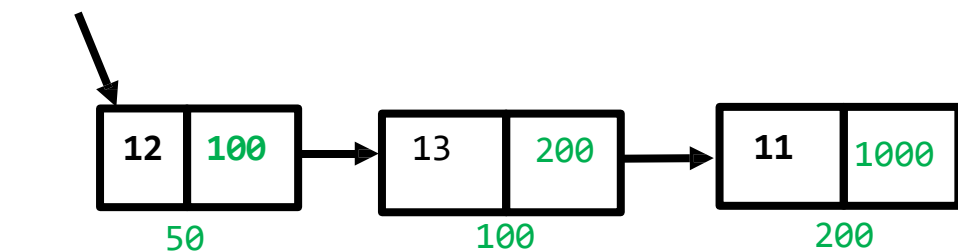


head

1000



head = 50



Inserting node at the middle of the List

```
struct Node *insertMid_SLL(struct Node *head)
```

```
{
```

```
    int newdata, loc=2, count=0;
```

```
    //Step-1. Create a newnode
```

```
    struct Node *newnode, *ptr;
```

```
    newnode = (struct Node*)malloc(sizeof(struct Node));
```

```
    //Step-2. Store the data
```

```
    printf("Enter an element to insert:");
```

```
    scanf("%d",&newnode->data);
```

```
    ptr=head;
```

```
    //Step-3. Traverse list, until required location
```

```
    while(ptr!= NULL && count!=loc)
```

```
    {
```

```
        ptr = ptr->next;
```

```
        count=count+1;
```

```
    }
```

```
    //Step-4. Move the next of prev_node as newnode
```

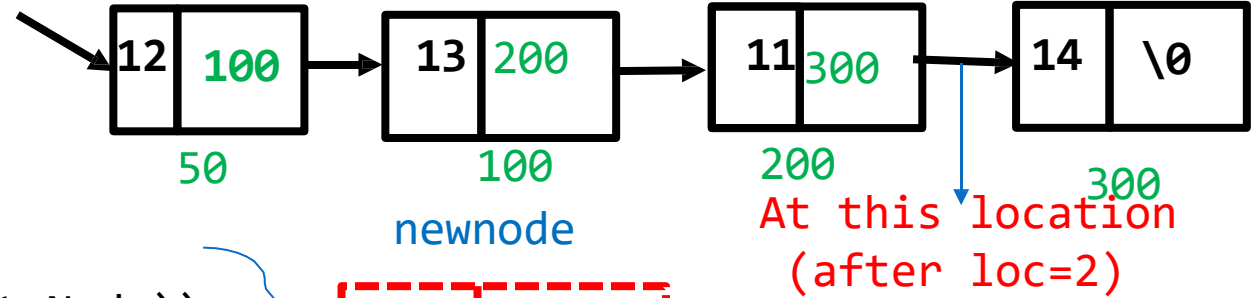
```
    newnode->next=ptr->next;
```

```
    ptr->next=newnode;
```

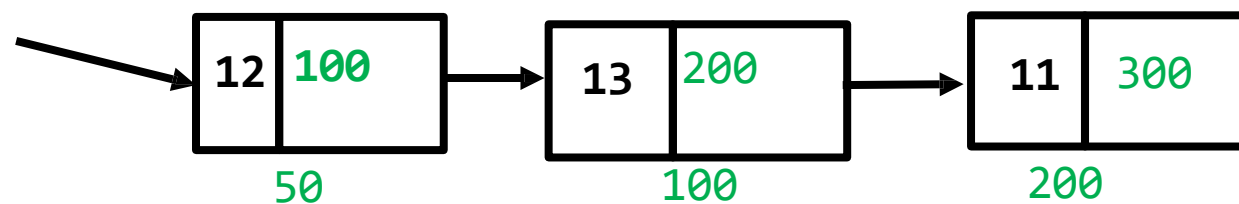
```
    return(head);
```

```
}
```

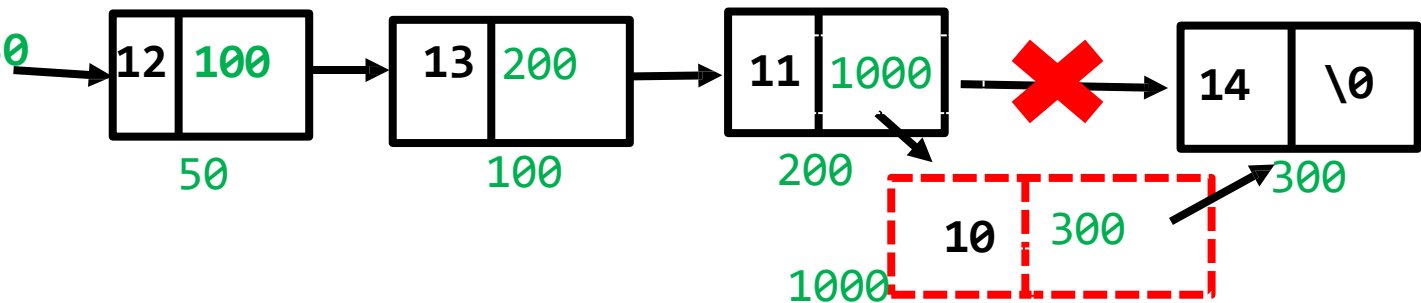
head = 50



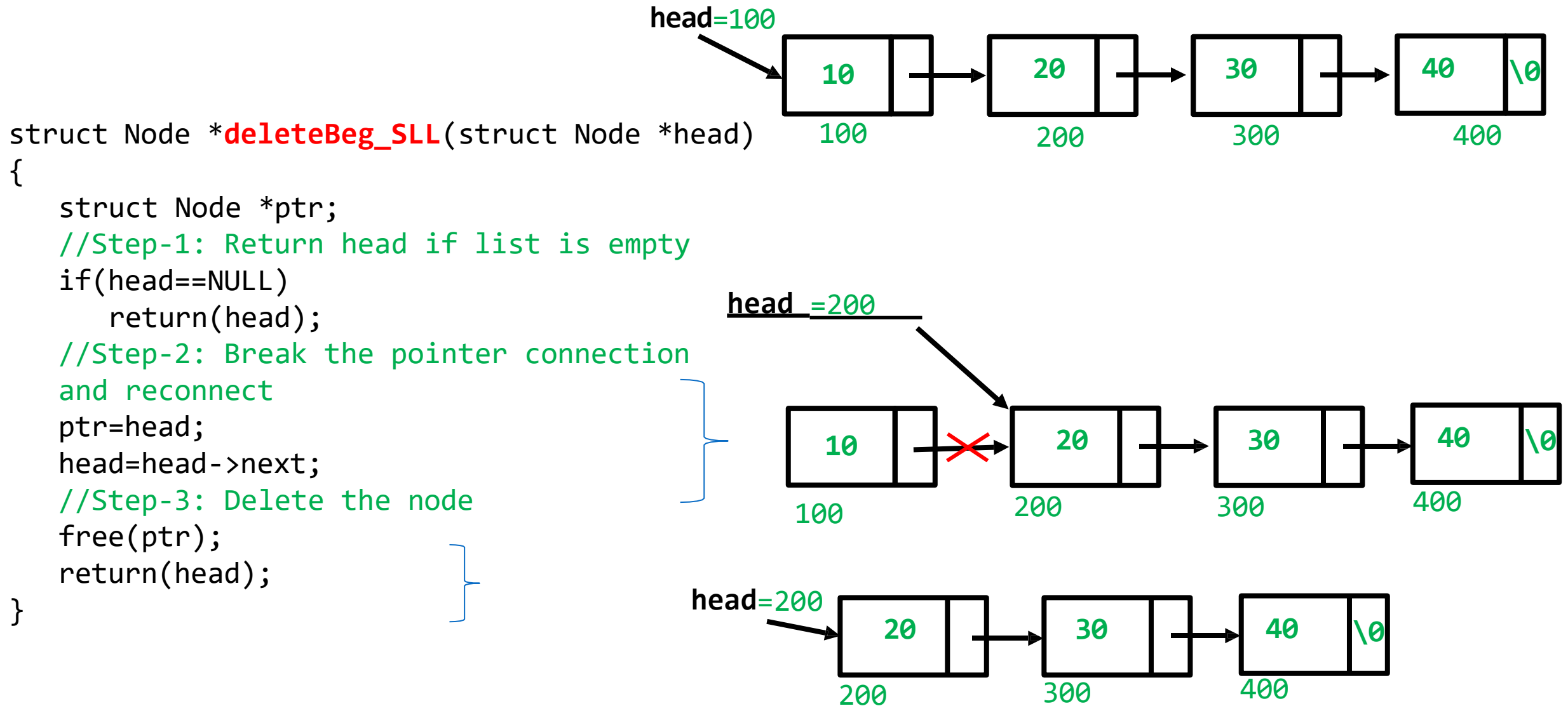
head = 50



head=50



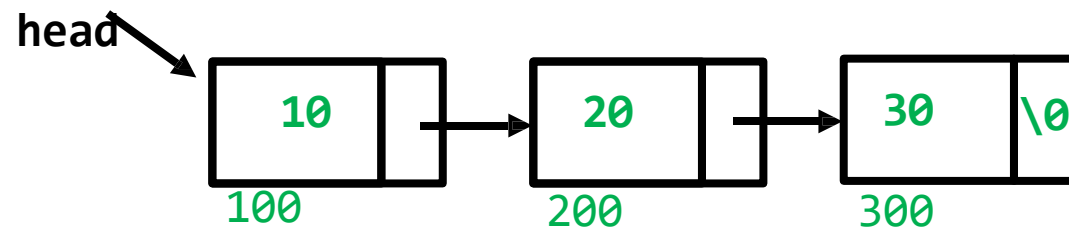
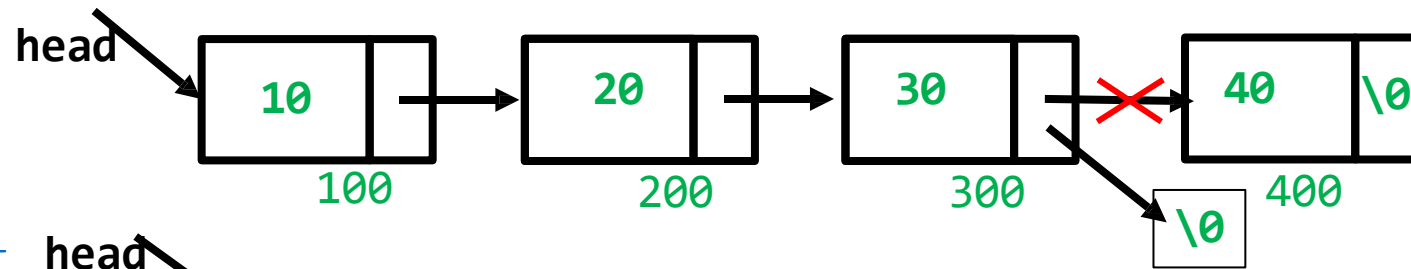
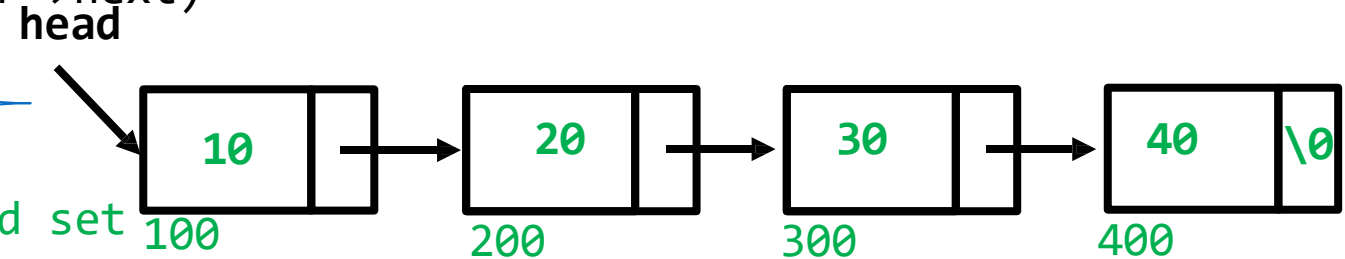
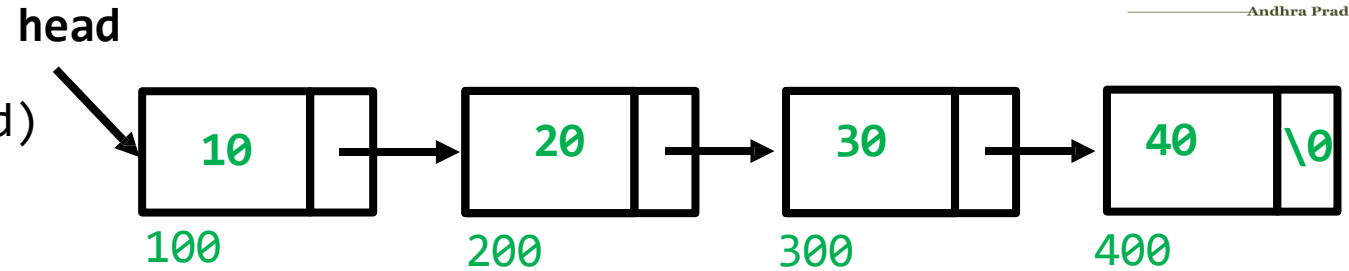
Delete node at beginning of Singly Linked List



Delete node at the end of Singly Linked List

```
struct Node *deleteEnd_SLL(struct Node *head)
{
    //Step-1: Return head if list is empty
    if(head==NULL)
        return(head);
    //Step-2: Traverse the list until end
    struct Node *curr, *prev;
    for(curr=head; curr->next!=NULL; curr=curr->next)
    {
        prev=curr;
    }
    //Step-3: Break the pointer connection and set
    //previous node pointer to null
    prev->next=NULL;

    //Step-2: Delete the node
    free(curr);
    return(head);
}
```



Delete node at the middle of Singly Linked List

```
struct Node *deleteMid_SLL(struct Node *head)
{
```

```
    int key;
    printf("Enter an element to delete:");
    scanf("%d",&key);
    struct Node *curr = head, *prev;
    if(curr == NULL)
```

```
        return(head);
```

```
    if(curr!= NULL && curr->data == key)
```

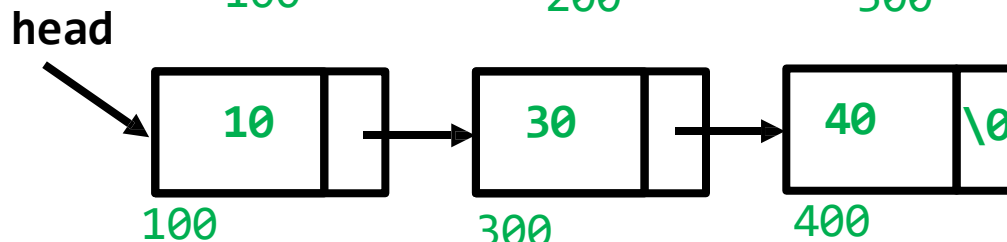
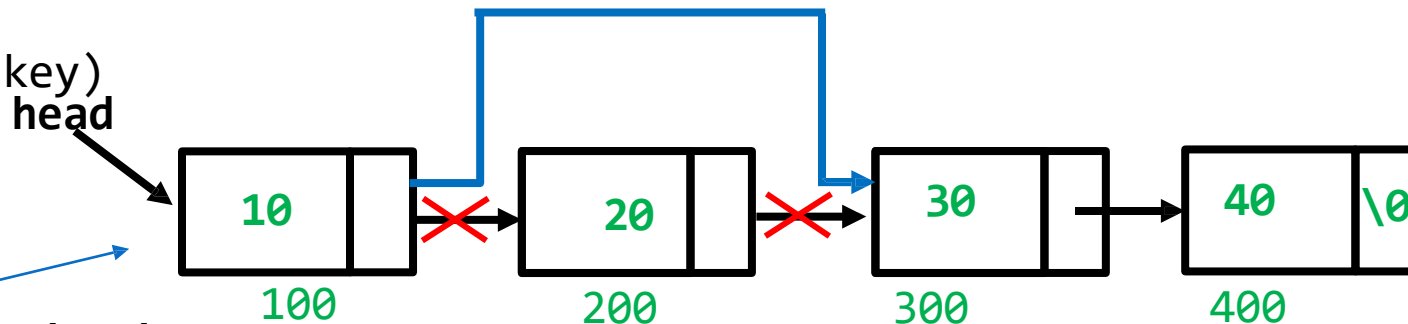
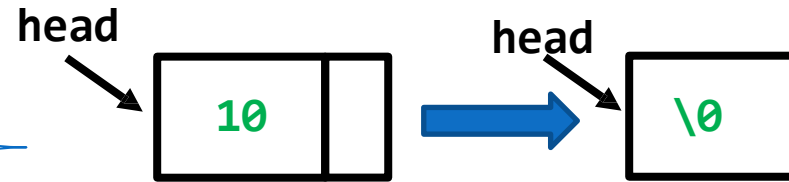
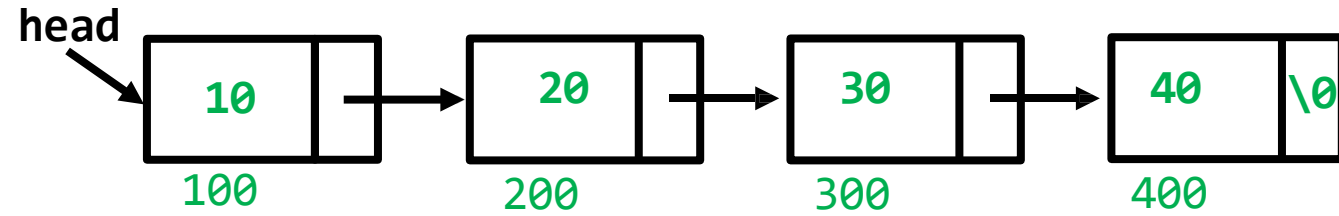
```
    {
        head= curr->next;
        free(curr);
        return(head);
```

```
    }
    while (curr != NULL && curr->data != key)
```

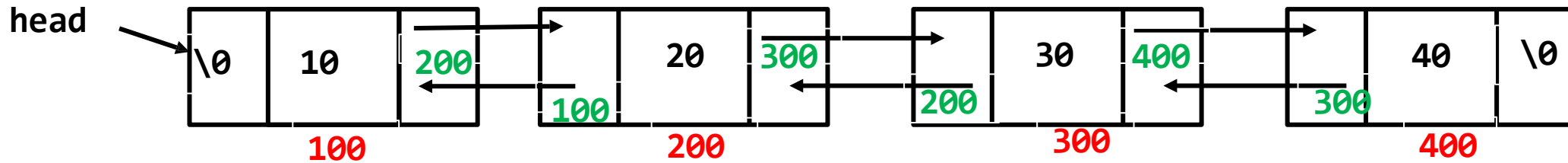
```
    {
        prev = curr;
        curr = curr->next;
```

```
    }
    prev->next = curr->next;
    free(curr);
```

```
}
```

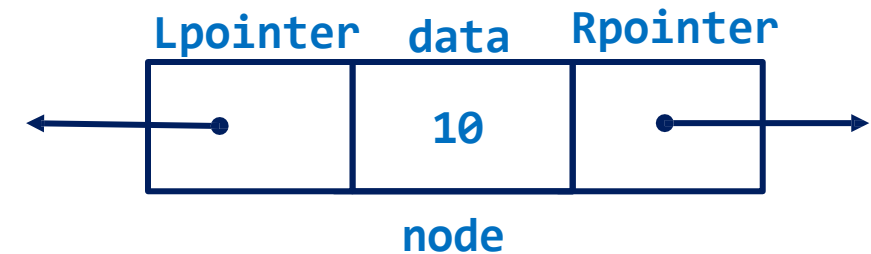


Doubly Linked List (DLL)



Advantages:

- Doubly linked list can be traversed forward and backward.
- Simplifies insertion and deletion because you no longer have to refer to the previous node.
- A node in a singly linked list cannot be removed unless we have the pointer to its predecessor.
- In a doubly linked list, we can delete a node even if we don't have the previous node's address (since each node has a left pointer pointing to the previous node and can move backward).



Disadvantage:

- Increase in space requirements.

Operations on Doubly Linked List (DLL)

- Empty Linked list is a single pointer having the value of NULL.

head = NULL;

- A node in DLL can be declared as:

```
struct Node
```

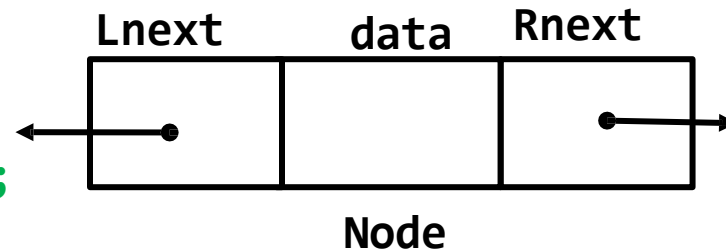
```
{
```

```
    struct Node *Lnext;
```

```
    int data;
```

```
    struct Node *Rnext;
```

```
};
```



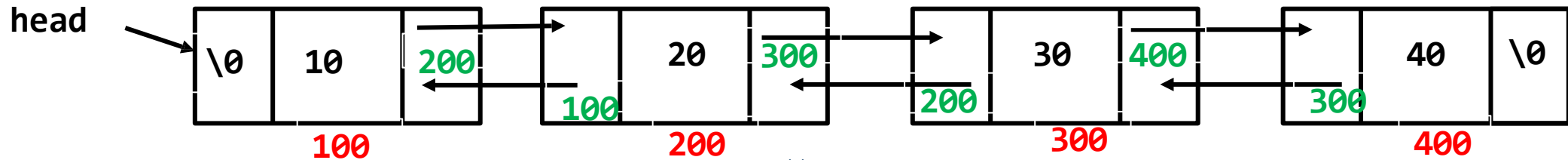
- A node in DLL can be created as:

```
head = (struct Node *)malloc(sizeof(struct Node));
```

Operations performed on DLL

- Create
- Traversing
- Search
- Insertion
 - At the begin
 - At the end
 - At the middle
- Deletion
 - At the begin
 - At the end
 - At the middle
- Display

Creation of the Doubly Linked List



```
struct Node
```

```
{
```

```
    struct Node *Lnext;
```

```
    int data;
```

```
    struct Node *Rnext;
```

```
};
```

```
void main()
```

```
{
```

```
    struct Node *head;
```

```
    head=create_DLL();
```

```
    display(head);
```

```
}
```

```
struct Node *create_DLL()
```

```
{
```

```
    int i, n;
```

```
    struct Node *curr, *temp, *head;
```

```
    printf("\n How many elements you want to enter?");
```

```
    scanf("%d", &n);
```

```
    for (i=1; i<=n; i++)
```

```
    {
```

```
        if (i == 1){
```

```
            head = (struct Node*)malloc(sizeof(struct Node));
```

```
            head->Lnext=NULL;
```

```
            curr = head;
```

```
        }
```

```
        else{
```

```
            temp = (struct Node*)malloc(sizeof(struct Node));
```

```
            curr->Rnext=temp;
```

```
            temp->Lnext=curr;
```

```
            curr=temp;
```

```
        }
```

```
        printf("Enter the %d th element: ", i)
```

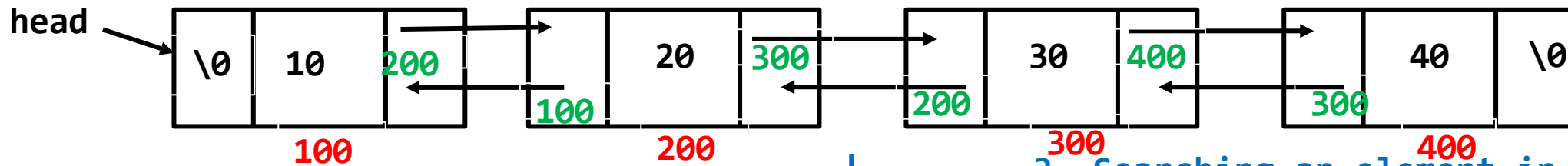
```
        scanf("%d",&curr->data);
```

```
    }
```

```
    curr->Rnext = NULL;
```

```
    return(head);
```

Search and Traverse operations on DLL



```
void main()
{
    struct Node *head;
    head=create_DLL();
    display_DLL(head);
    search_DLL(head);
}
```

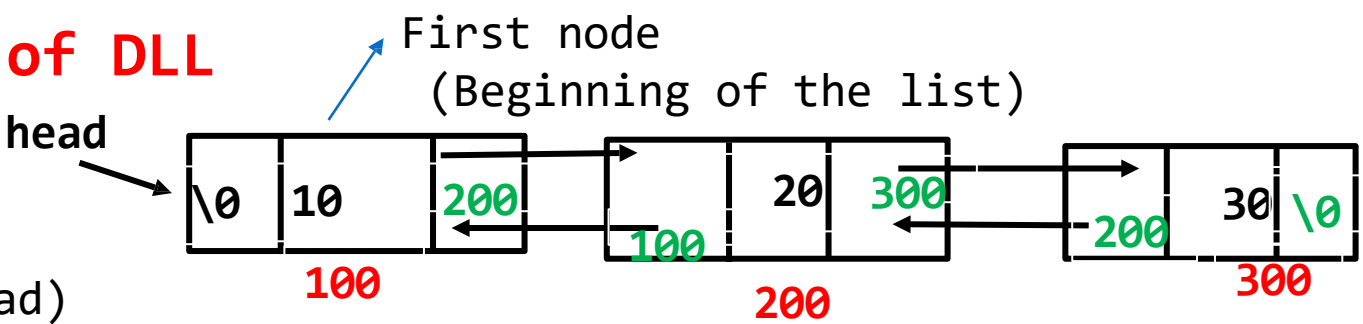
2. Traversing a DLL

```
void display_DLL(struct Node *head)
{
    struct Node *ptr;
    int count=0;
    for(ptr=head; ptr!= NULL; ptr = ptr->next)
    {
        printf ("\n Node data= %d", ptr->data);
    }
    printf ("\n");
}
```

3. Searching an element in DLL

```
void search_DLL(struct Node *head)
{
    struct Node *ptr;
    int key, count=0;
    printf("Enter an element to search:");
    scanf("%d",&key);
    for(ptr = head; ptr!= NULL; ptr = ptr->next)
    {
        if ((ptr->data)==key)
        {
            count=1;
            break;
        }
    }
    if(count==1)
        printf("found");
    else
        printf("not found");
}
```

Inserting node at the beginning of DLL



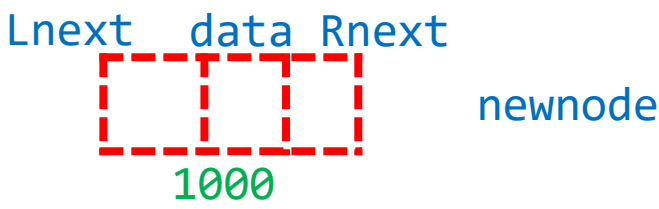
```
struct Node *insertBeg_DLL(struct Node *head)
{
    //Step-1. create a new node
    struct Node* newnode;
    newnode = (struct Node*) malloc(sizeof(struct Node));

    //Step-2. Store the data

    printf("Enter an element to insert at beginning:");
    scanf("%d",&newnode->data);

    //Step-3. Make next of new node as head
    newnode->Rnext = head;
    newnode->Lnext = NULL;
    head->Lnext = newnode;

    //Step-4. Move the head to point to the new node
    head=newnode;
    return(head);
}
```



Inserting node at the end of DLL

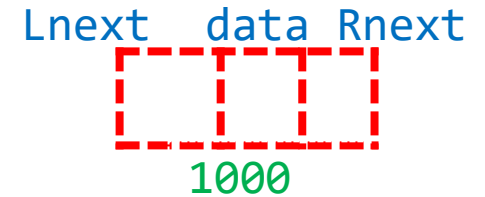
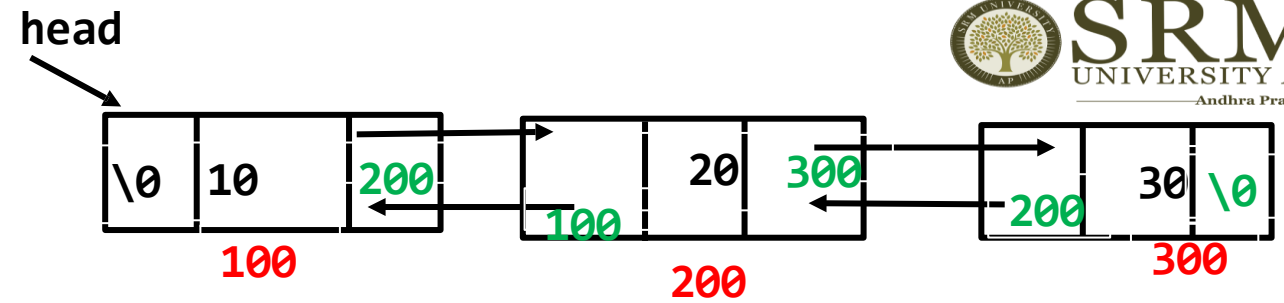
```

struct Node *insertEnd_DLL(struct Node *head)
{
    //Step-1. Create new node
    struct Node *newnode, *ptr;
    newnode = (struct Node*) malloc(sizeof(struct Node));

    //Step-2. Store the data
    printf("Enter an element to insert:");
    scanf("%d",&newnode->data);
    //Step-3. New node Should be inserted last
    // so make next of it as NULL
    newnode->Rnext = NULL;

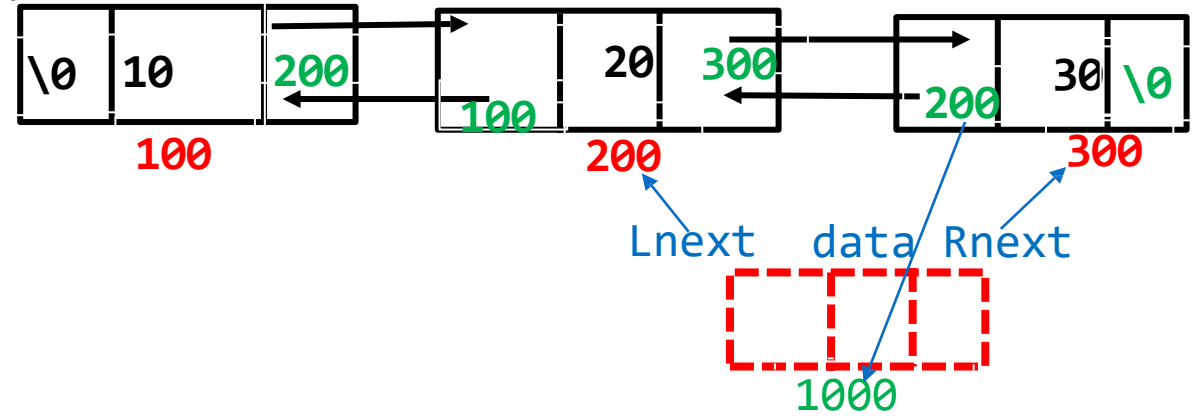
    //Step-4. If list is empty, then make newnode as head
    if (head == NULL){
        newnode->Lnext = NULL;
        head = newnode;
        return(head);
    }
    //Step-5. Else traverse till the last node
    ptr=head;
    while (ptr->Rnext != NULL){
        ptr = ptr->Rnext;
    }
    //Step-6. Change the next of last node
    ptr->Rnext = newnode;
    newnode->Lnext = ptr;
    return(head);
}

```



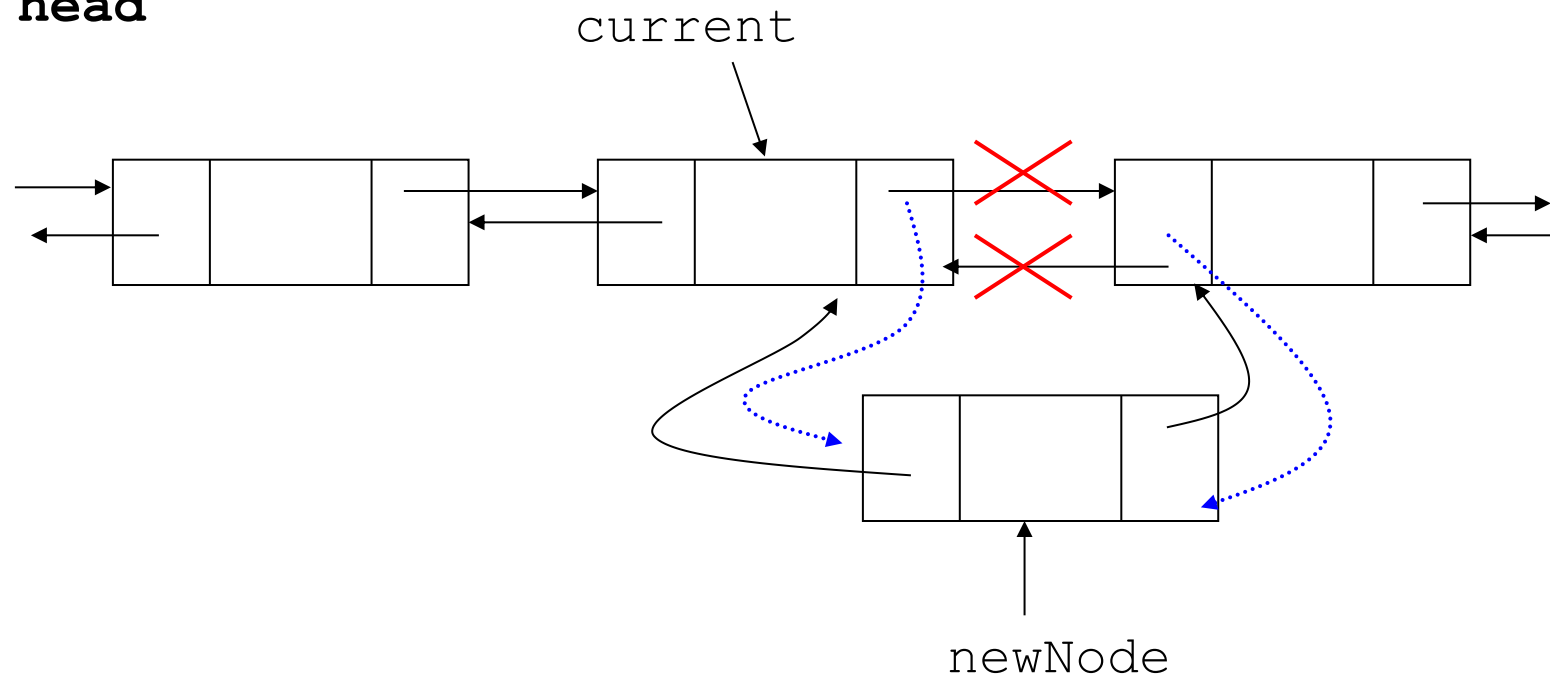
Inserting node at the middle of DLL

head



```
newNode = (struct Node*)malloc(sizeof(struct Node));
newNode->Rnext = current->Rnext;
(current->Rnext)->Lnext=newnode
newNode->Lnext = current;
current->Rnext = newNode;
current = newNode;
```

head



Delete node at beginning of DLL

```
struct Node *deleteBeg_DLL(struct Node *head)
```

```
{
```

```
    // Check if the list is empty
```

```
    if (head == NULL) return NULL;
```

```
    temp = head; // Store reference to the current head
```

```
    head = head->Rnext; // Move head to the next node
```

```
    // If the new head is not NULL, update its Lnext to NULL
```

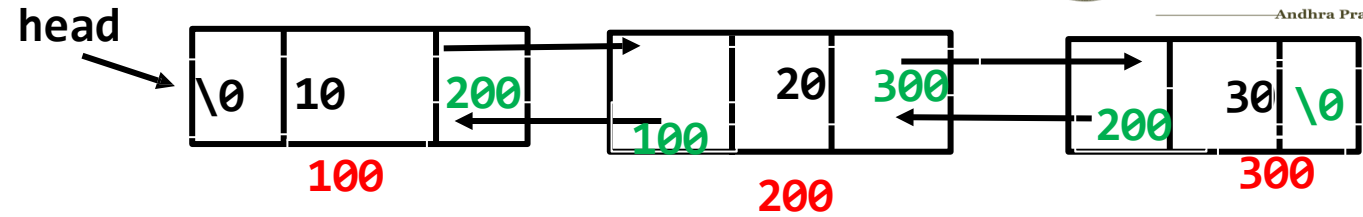
```
    if (head != NULL)
```

```
        head->Lnext = NULL;
```

```
    free(temp); // Free the old head
```

```
    return head; // Return the updated head
```

```
}
```

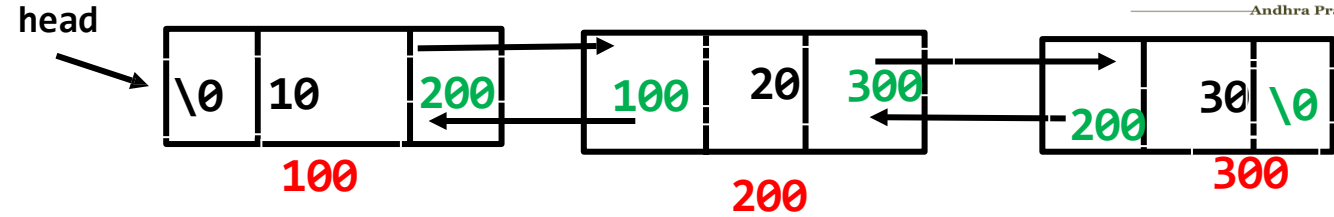


Delete node at the end of DLL

```

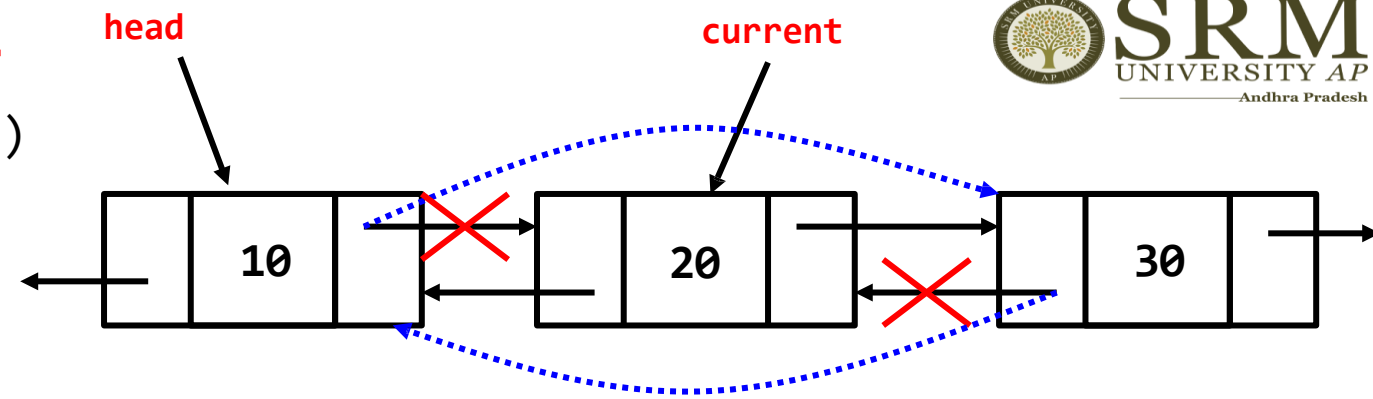
struct Node *deleteEnd_DLL(struct Node *head)
{
    // Check if the list is empty
    if (head == NULL) return NULL;
    curr = head; // Initialize curr to head

    // Traverse to the last node
    while (curr->Rnext != NULL) {
        curr = curr->Rnext;
    }
    // If there is only one node in the list
    if (curr->Lnext == NULL) {
        free(curr); // Free the only node
        return NULL; // Return NULL since the list is now empty
    }
    else {
        // Update the second last node's Rnext to NULL
        curr->Lnext->Rnext = NULL;
        free(curr); // Free the last node
    }
    return head; // Return the updated head
}
  
```



Delete node at the middle of DLL

```
struct Node *deleteMid_DLL(struct Node *head)
{
    printf("Enter an element to delete:");
    scanf("%d",&key);
    curr=head;
    if(curr == NULL)
        return(head);
    if(curr!= NULL && curr->data == key){
        head= curr->Rnext;
        free(curr);
        return(head);
    }
    while (curr != NULL && curr->data != key){
        curr = curr->Rnext;
    }
}
```



// If the key was not found, simply return the original list.

```
if (curr == NULL)
```

```
    return head;
```

// Reconnect the previous and next nodes using curr's own pointers.

```
if (curr->Lnext != NULL)
```

```
    curr->Lnext->Rnext = curr->Rnext;
```

```
if (curr->Rnext != NULL)
```

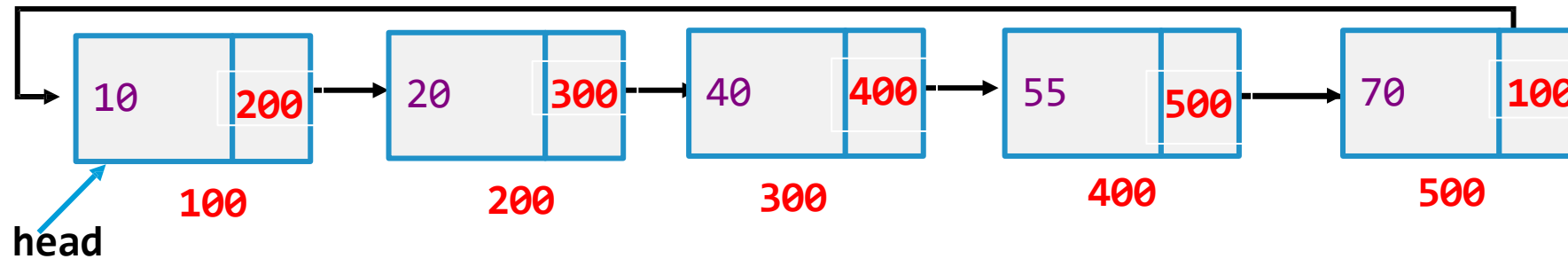
```
    curr->Rnext->Lnext = curr->Lnext;
```

```
free(curr);
```

```
return head;}
```

Circularly Linked Lists (CLL)

- It supports traversing from the end of the list to the beginning by making the last node point back to the head of the list.



```
struct Node
{
    int data;
    struct Node *next;
};
```

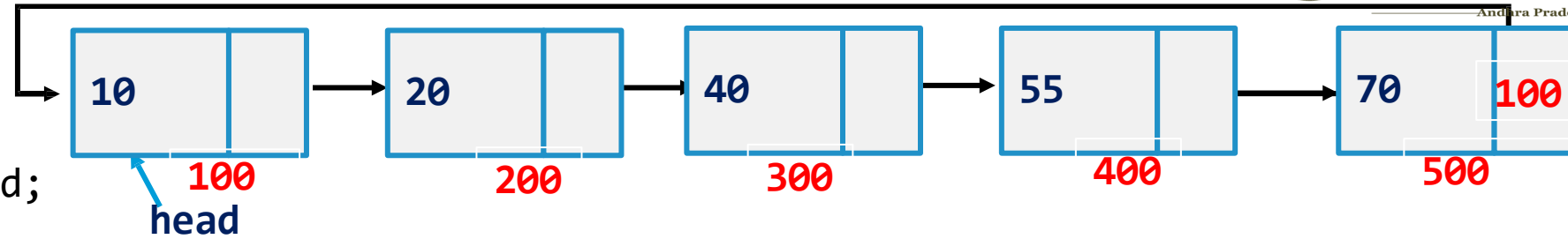
Operations on Circularly Linked Lists (CLL)

```
struct Node *create_CLL()
{
    int i, n;
    struct Node *ptr, *head;

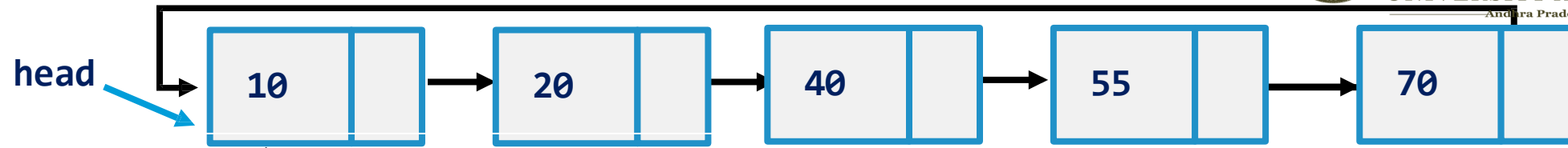
    printf("\n How many elements to enter?");
    scanf("%d", &n);

    for (i=1; i<=n; i++)
    {
        if (i == 0)
        {
            head = (struct Node*)malloc(sizeof(struct Node));
            ptr = head;
        }
        else
        {
            ptr->next = (struct Node*)malloc(sizeof(struct Node));
            ptr = ptr->next;
        }
        printf("Enter the value of %d node: ", i);
        scanf("%d",&ptr->data);
    }

    ptr->next = head;
    return(head);
}
```



Operations on Circular Linked Lists (CLL)



```
void search_CLL(struct Node *head)
{
```

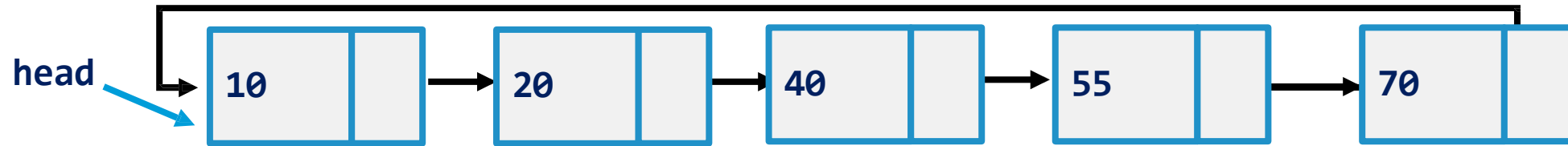
```
    struct Node *ptr;
    int key, count=0;
    printf("Enter an element to search:");
    scanf("%d",&key);
    if(head->data==key){
        printf("Found"); return; }
    for(ptr = head->next; ptr!= head; ptr = ptr->next)
    {
        if ((ptr->data)==key){
            count=1;
            break;
        }
    }
    if(count==1)
        printf("found");
    else
        printf("not found");
```

```
}
```

Data Structures--> Unit-II:

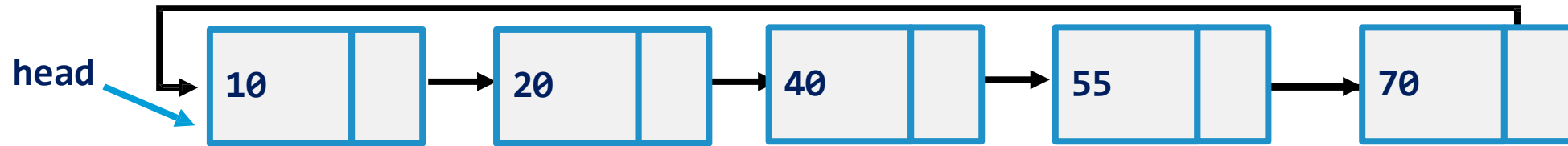
```
void display_CLL(struct Node *head)
{
    struct Node *ptr;
    printf("\nNode data %d", head->data);
    for(ptr=head->next; ptr!= head; ptr = ptr->next)
    {
        printf("\nNode data=%d", ptr->data);
    }
    printf ("\n");
}
```

Inserting node at beginning of CLL



```
newnode->next = head;
ptr=head;
while(ptr->next !=head)
{
    ptr=ptr->next;
}
ptr->next=newnode
head=newnode;
return(head);
```

Inserting node at Middle of CLL

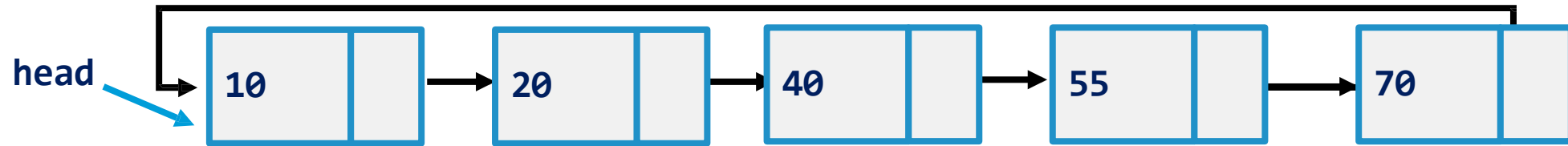


```
ptr=head;
while(ptr!= head && count!=loc)
{
    ptr = ptr->next;
    count=count+1;
}
newnode->next=ptr->next;
ptr->next=newnode;
return(head);
```

Delete node at beginning of CLL

```
struct node* delete_beginning(struct node *head) {  
    struct node *temp, *ptr;  
    if(head == NULL)          // Empty list  
        return NULL;  
    if(head->next == head) {    // Single node list  
        free(head);  
        return NULL;  
    }  
    ptr = head;  
    while(ptr->next != head)    // Find last node  
        ptr = ptr->next;  
    temp = head;  
    head = head->next;          // Update head  
    ptr->next = head;           // Link last node to new head  
    free(temp);                // Free old head  
    return head;  
}
```

Delete node at middle of CLL



```
struct Node *deleteMid_CLL(struct Node *head) {
```

```
    int key;
```

```
    printf("Enter an element to delete:");
```

```
    scanf("%d", &key);
```

```
    struct Node *curr = head, *prev = NULL;
```

```
    if (head == NULL) return head;
```

```
    if (head->data == key && head->next == head) {
```

```
        free(head);
```

```
        return NULL;
```

```
    }
```

```
    while (curr->next != head && curr->data != key) {
```

```
        prev = curr;
```

```
        curr = curr->next;
```

```
    }
```

```
    if (curr->data == key) {
```

```
        prev->next = curr->next;
```

```
        free(curr);
```

```
    }
```

```
    return head;
```

```
}
```